Лабораторная работа №3

Моделирование графов атак

Аристова Арина Олеговна

2026-05-02

Докладчик

- Аристова Арина Олеговна
- студентка группы НФИмд-01-25
- Российский университет дружбы народов
- 1032259382@rudn.ru
- https://github.com/aoaristova

Цель работы

- Освоить методы построения и анализа графов атак для оценки уязвимостей сетевой инфраструктуры.
- На примере моделирования атак на корпоративную сеть изучить:
  - представление сетевой топологии и уязвимостей в виде ориентированного графа;
  - алгоритмы поиска всех возможных путей атаки от начальных точек до целевых активов;
  - расчёт метрик центральности для определения критических узлов;
  - визуализацию графа с цветовой индикацией уровня риска;
  - оценку вероятности успешной атаки с учётом сложности эксплуатации уязвимостей.

Задание

- Построить граф атак для заданной топологии сети.
- Реализовать алгоритм поиска всех путей от заданного источника к цели.
- Рассчитать метрики центральности для всех узлов и выявить наиболее критичные.
- Визуализировать граф, раскрашивая узлы в зависимости от степени риска
- Присвоить каждому ребру вес (вероятность успешной атаки) и вычислить наиболее вероятный путь атаки

Раздел 1

Выполнение лабораторной работы

Ядро моделирования графа атак

Содержит все функции для построения ориентированного графа атак, поиска путей, расчёта метрик центральности, присвоения весов и нахождения наиболее вероятного пути. Это основной модуль, используемый всеми скриптами.

Ядро моделирования графа атак

add_packages.jl | attack_graph.jl | ag_run_experiment.jl

project > src > attack_graph.jl

```julia
using Graphs, LinearAlgebra
"""
build_attack_graph(n, edge_prob, vulnerabilities,
trust_relations)
Создаёт ориентированный граф атак с заданными параметрами.
"""
function build_attack_graph(n, edge_prob, vulnerabilities,
    g = SimpleDiGraph(n)
    # случайные рёбра
    for i = 1:n, j = 1:n
        if i != j && rand() < edge_prob
            add_edge!(g, i, j)
        end
    end
    # доверительные отношения
    for (u, v) in trust_relations
        add_edge!(g, u, v)
    end
    return g
end
```

Рисунок 1: Файл attack_graph.jl

Однократный запуск эксперимента

Файл scripts/ag_run_experiment.jl выполняет построение графа атак для заданных параметров, находит все пути, вычисляет метрики центральности, определяет наиболее вероятный путь и со- храняет результаты в JLD2-файл. Предназначен для генерации данных, которые будут анализироваться в других скриптах

Однократный запуск эксперимента

add_packages.jl | attack_graph.jl | ag_run_experiment.jl

project > scripts > ag_run_experiment.jl

```julia
using DrWatson
@quickactivate "project"
using Graphs, JLD2, Random
include(srcdir("attack_graph.jl"))

params = Dict(
    :n => 20,
    :edge_prob => 0.2,
    :source => 1,
    :target => 20,
    :cvss_scores => Dict((1, 3) => 0.9, (2, 5) => 0.7, (3, 8
    :trust_relations => [(4, 6), (6, 10), (10, 15)],
)
filename = datadir("attack_graph", savename(params, "jld2"))
mkpath(datadir("attack_graph"))
if isfile(filename)
    @load filename data
    println("Данные загружены из $filename")
else
    g = build_attack_graph(
```

Рисунок 2: Файл ag_run_experiment.jl

Однократный запуск эксперимента

Результат:

В папке data/attack_graph/ создаётся файл. В нём хранятся:

- :graph - объект графа;
- :paths - массив всех путей;
- :metrics - словарь с метриками центральности;
- :weights - веса рёбер;
- :likely_path - массив узлов наиболее вероятного пути;
- :probability - числовая вероятность успеха

Однократный запуск эксперимента

```
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab03\project> julia .\scripts\ag_run_experiment.jl
  Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab03\project`
Результаты сохранены в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab03\project\data\attack_graph\edge_
prob=0.2_n=20_source=1_target=20.jld2
Количество путей атаки: 6336
Наиболее вероятный путь: [1, 14, 5, 20]
Вероятность успеха: 0.2375
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab03\project>
```

Рисунок 3: Результат выполнения ag_run_experiment.jl в консоли

Однократный запуск эксперимента

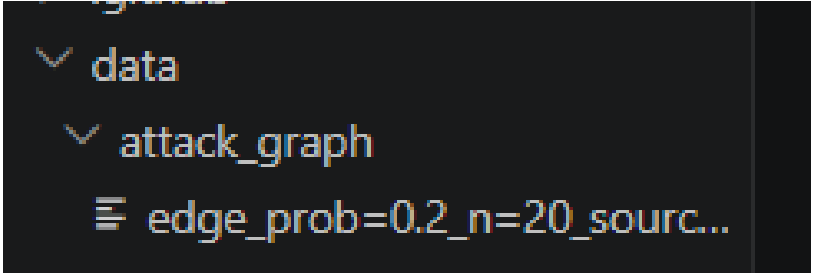

Рисунок 4: Сохранённый файл - результат выполнения ag_run_experiment.jl в консоли

Визуализация и анализ графа

Файл scripts/ag_analyze.jl загружает сохранённые данные. Строит наглядное изображение графа (с цвето- вой индикацией по PageRank). Выводит статистику: количество узлов, рёбер, пути, топ-5 критичных узлов по in-degree и PageRank.

l_packages.jl | attack_graph.jl | ag_run_experiment.jl | ag_analyze.jl

project > scripts > ag_analyze.jl

```julia
using DrWatson
@quickactivate "project"
using Graphs, JLD2, Plots, GraphRecipes
include(srcdir("attack_graph.jl"))

params = Dict(:n => 20, :edge_prob => 0.2, :source => 1, :target
filename = datadir("attack_graph", savename(params, "jld2"))
@load filename data
g = data[:graph]
metrics = data[:metrics]
pagerank = metrics[:pagerank]
norm_rank = (pagerank .- minimum(pagerank)) ./ (maximum(pagerank
colors = [cgrad(:RdYlGn, rev = true)[norm_rank[i]] for i = 1:nv(
graphplot(
    g,
    nodeshape = :circle,
    curves = false,
    linecolor = :black,
    nodecolor = colors,
    nodelabel = 1:nv(g),
    title = "Граф атак (цвет = PageRank)",
    size = (800, 600),
)
```

Рисунок 5: Файл ag_analyze.jl

Визуализация и анализ графа

Результат:

- PNG-файл с визуализацией графа в папке plots/.
- Текстовый отчёт в консоли.

Визуализация и анализ графа

```
]
Наиболее вероятный путь: [1, 14, 5, 20]

Топ-5 узлов по in-degree:
 Узел 9: in-degree = 8
 Узел 15: in-degree = 8
 Узел 4: in-degree = 7
 Узел 16: in-degree = 7
 Узел 6: in-degree = 6

Топ-5 узлов по PageRank:
 Узел 16: PageRank = 0.0912
 Узел 9: PageRank = 0.0906
 Узел 15: PageRank = 0.0844
 Узел 4: PageRank = 0.0783
 Узел 1: PageRank = 0.0747
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab03\project>
```

Рисунок 6: Результат выполнения ag_analyze.jl в консоли

Визуализация и анализ графа

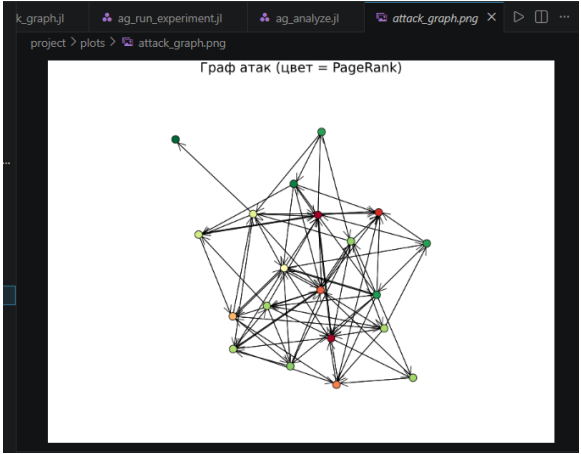

Рисунок 7: Полученный график - результат выполнения ag_analyze.jl

Исследование масштабируемости

Файл scripts/ag_convergence.jl исследование, как размер сети (число узлов) влияет на время поиска всех путей и на количество найденных путей. Позволяет оценить вычислительную сложность алгоритма.

attack_graph.jl | ag_run_experiment.jl | ag_analyze.jl | attack_graph.png | ag_convergence.jl

project > scripts > ag_convergence.jl

```julia
using DrWatson
@quickactivate "project"
using Graphs
using Plots
using JLD2
using Random

include(srcdir("attack_graph.jl"))
sizes = [10, 15, 20, 21, 22, 23, 24, 25]
time_vals = Float64[]
path_counts = Int[]
Random.seed!(123)
for n in sizes
    println("Размер сети: $n")
    g = build_attack_graph(n, 0.2, Dict(), [])
    t = @elapsed paths = find_all_paths(g, 1, n)
    push!(time_vals, t)
    push!(path_counts, length(paths))
    println(" Время: $(round(t, digits=3)) с, путей: $(length(paths))")
end
p1 = plot(
    sizes,
    time_vals,
    marker = :circle,
```

OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | powershell - project

Рисунок 8: Файл ag_convergence.jl

Исследование масштабируемости

Результат:

- График convergence.png, показывающий рост времени и числа путей с увеличением сети
- JLD2 файл с данными для возможного повторного использования.

Исследование масштабируемости

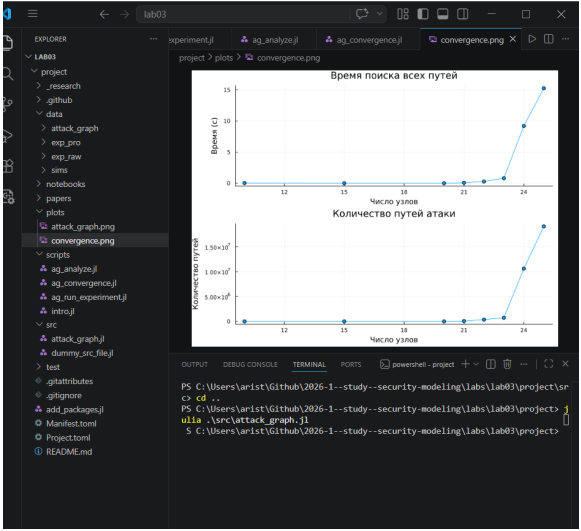

Рисунок 9: Полученный график - результат выполнения convergence.jl

Исследование масштабируемости

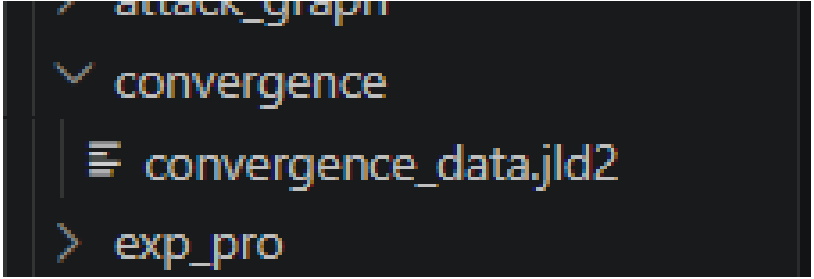

Рисунок 10: Сохранённый файл - результат выполнения convergence.jl в консоли

Параметрическое исследование

Файл scripts/parameter_sweep.jl изучает, как изменение плотности рёбер (вероятности случайного ребра) влияет на количество путей атаки, максимальную входящую степень и среднюю длину пути.

e.jl | ag_convergence.jl | ag_convergence_2.jl | parameter_sweep.jl

project > scripts > parameter_sweep.jl

```julia
using DrWatson
@quickactivate "project"
using Graphs, Random, CSV, DataFrames, Plots, StatsBase
include(srcdir("attack_graph.jl"))
edge_probs = 0.1:0.1:0.9
n_nodes = 13
source = 1
target = n_nodes
results = []
Random.seed!(123)
for p in edge_probs
    println("Плотность рёбер: $p")
    g = build_attack_graph(n_nodes, p, Dict(), [])
    paths = find_all_paths(g, source, target)
    metrics = compute_centrality_metrics(g)
    max_indeg = maximum(metrics[:in_degree])
    avg_path_len = isempty(paths) ? 0.0 : mean(length.(paths))
    push!(
        results,
```

Рисунок 11: Файл parameter_sweep.jl

Параметрическое исследование

Результат:

- Таблица результатов в CSV.
- Графики (рис. 3.3), показывающие, как увеличение связности графа (плотно- сти) приводит к росту числа путей и их средней длины.

Параметрическое исследование

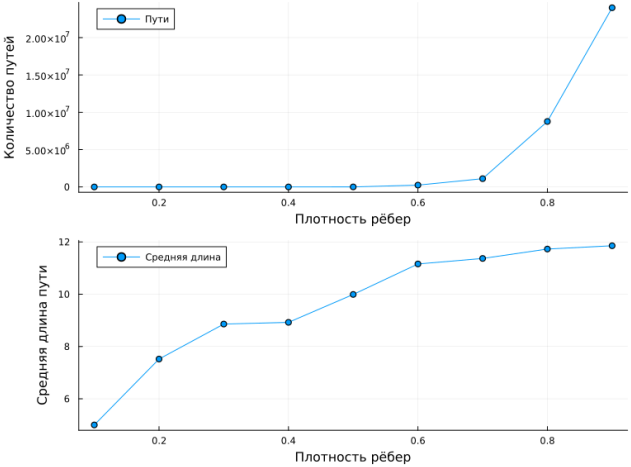

Рисунок 12: Полученный график - результат выполнения parameter_sweep.jl

Параметрическое исследование

```
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab03\project> julia .\scripts\parameter_sweep.jl
  Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab03\project`
Плотность рёбер: 0.1
Плотность рёбер: 0.2
Плотность рёбер: 0.3
Плотность рёбер: 0.4
Плотность рёбер: 0.5
Плотность рёбер: 0.6
Плотность рёбер: 0.7
Плотность рёбер: 0.8
Плотность рёбер: 0.9
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab03\project>
```

Рисунок 13: Сохранённый файл - результат выполнения parameter_sweep.jl в консоли

Выводы

В результате данной лабораторно работы освоены методы построения и анализа графов атак для оценки уязвимостей сетевой инфраструктуры, а также выполнен пример моделирования атак на корпоративную сеть.

Список литературы

1. Описание лабораторной работы